Problem Definition & Requirements

A) Problem Statement

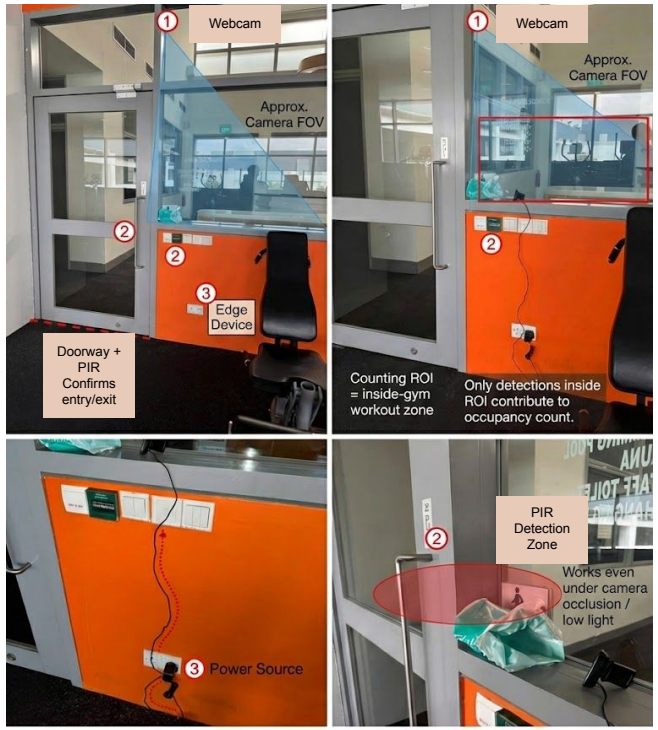
- **Problem:** Condo residents waste time due to uncertain gym crowd + equipment availability (peak hours), causing unnecessary waiting / repeated trips.
- **Need:** Provide near real-time status of (1) gym crowd level and (2) key equipment zones (in-use vs idle) so residents can plan visits.
- **Approach (one line only):** Use on-site camera + PIR sensors to infer occupancy/zone usage and send metadata only to a dashboard (no raw video uplink).

B) Why this is an Edge Computing problem

- **Real-time constraint:** stale updates reduce usefulness (target **sub-1s**, ideal **≤ 0.5s**)
- **Sensor-driven:** camera frames + PIR motion events
- **Privacy boundary: no raw video/images leave site**
- **Intermittent connectivity:** must work on **local LAN even if internet is down**
- **Decision loop not cloud-dependent:** inference happens **on edge devices / edge server**
- **Energy constraint:** always-on deployment → keep compute **low-power** (power budget per node)

C) Real-time + System Requirements

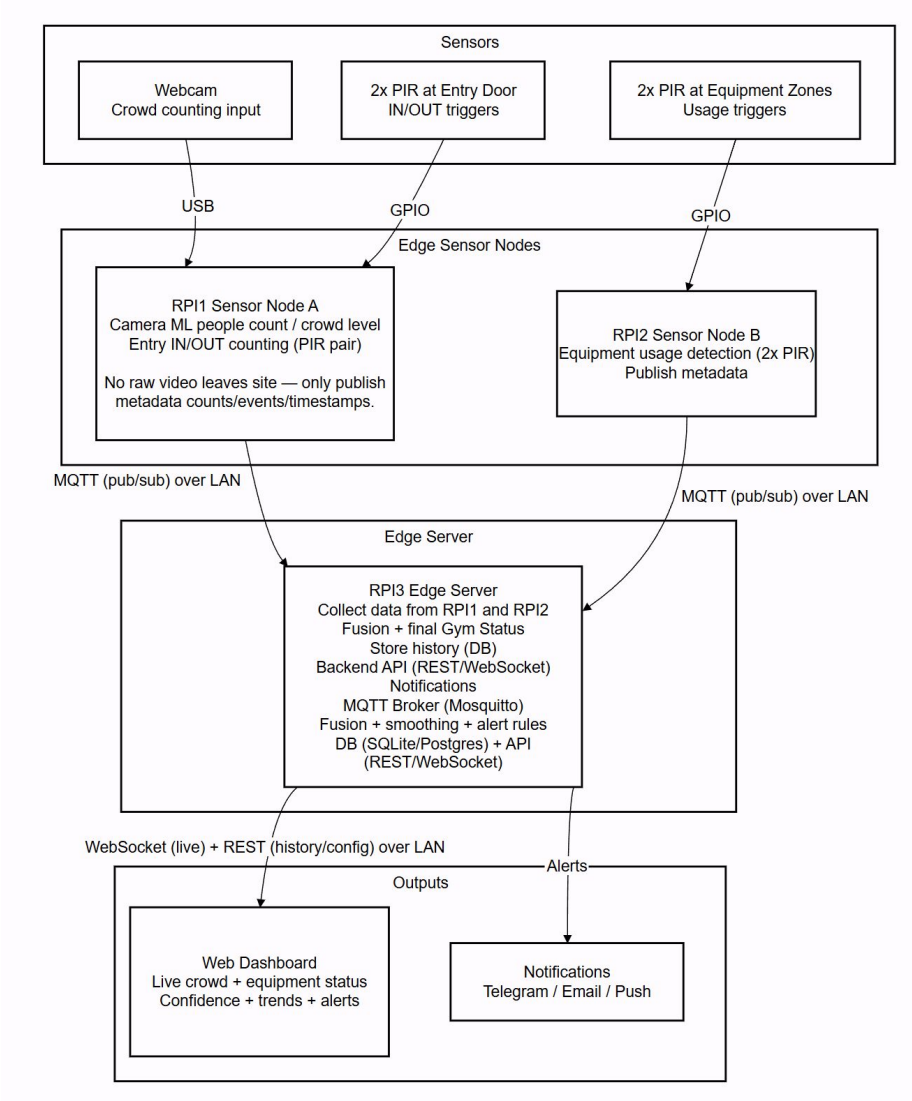
Requirement	Target (numeric)	Measured / Evidence	Why it matters
On-device vision loop (frame → inference → ROI logic)	**p95 ≤ 300 ms** (/min ≥ 4 FPS)	**p95 ≈ 307 ms**, avg ≈ 244 ms (**~4.1 FPS**) on RPi 5	Keeps crowd status "fresh"
End-to-end update (sensor → dashboard)	**≤ 1.0 s** (ideal ≤ 0.5 s)	Budgeted (to be measured after publish/receive instrumentation)	Residents need near real-time view
Bandwidth	**≤ 0.2 Mbps avg** (≈ 25 KB/s)	Send **counts/events only** (e.g., 1 Hz or on-change)	Avoid streaming video cost
Power / Energy (24/7 operation)	**≤ 10 W avg per RPi node** (system ≤ 25 W avg)	**Power budget target** (will verify with USB-C inline power meter under load)	Lower operating cost + less heat; feasible to run continuously in condo facility
Privacy	No raw video/images off-site	Only **aggregated counts/events** stored/transmitted	PDPA/trust requirement
Availability / offline	Works with **0 internet on LAN** + buffer **≥24h**	Edge nodes continue inference + local buffering	Still usable during outages



System Architecture Diagram

Data flow, where inference runs, and fallback behaviour.

A) Architecture Diagram



B) What runs where (edge-first) + fallback

What runs where (edge-first)

- **RPi1 (Vision + Entry PIR):** YOLOv8n detect+track @ ~4–5 FPS (480p) + entry PIR IN/OUT → publish **metadata only** (count, crowd_level, entry/exit).
- **RPi2 (Equipment PIR):** PIR state machine @ 10 Hz → publish **zone busy/idle + timestamps**.
- **RPi3 (Edge server): MQTT broker** + fusion/smoothing + DB + REST/WebSocket + alert rules → dashboard/notifications.

Why MQTT (telemetry bus)

- **Frequent small updates:** optimized for lightweight event/state messages (lower overhead than repeated REST).
- **Decoupled & scalable:** pub/sub lets us add dashboard/logger/alerts by subscribing, without changing sensor nodes.
- **Robust on flaky Wi-Fi:** supports **QoS + retained latest state + LWT** (node offline detection).

Fallback / offline behaviour

- **Camera slow/fail:** reduce FPS/resolution → fallback to **PIR-only approximate occupancy**.
- **RPi3/LAN down:** RPi1/RPi2 keep sensing + **buffer locally (file/SQLite)** → replay when back.
- **Internet down:** dashboard still works on **LAN (offline-first)**; only remote alerts may stop.

Data policy

- **No raw video leaves site**. Only counts/zone status/timestamps/trends.

Problem Decomposition + Model Mapping + Model Selection Justification

A) Decomposition + model mapping

Sub-task (pipeline)	Input → Output	Runs where	Method
Trigger / gating	PIR motion / IN-OUT edges + "ROI enabled" → run_inference, entry_event, exit_event	*RPi1/RPi2*	Rule-based PIR state machine (10Hz)
Main inference	Camera frame (480p, rate-limited) → person boxes + IDs	*RPi1*	YOLOv8n (person) + tracker
Post-processing	boxes/IDs + ROI polygon → people_in_ROI, dwell_time, crowd_level_raw	*RPi1*	ROI membership + dwell/grace anti-flicker
Sensor interpretation (equipment zones)	Zone PIR signals → zone_in_use, zone_idle, timestamps	*RPi3*	Fusion rules + smoothing (sanity-check/fallback)
Fusion / decision	camera count + entry IN/OUT + zone states → final occupancy, Low/Med/High crowd, final equipment status	*RPi3*	WebSocket/REST + alert rules + DB

Edge-first: only metadata/events transmitted; no raw video.

B) Model selection justification

Candidate	Format / size	Speed on RPi5	Accuracy proxy	Decision
YOLOv8n (chosen)	~6–8MB (FP16 .pt)	*Measured: loop avg 244ms, p95 307ms (~4 FPS)*	Good indoor person detect	Meets real-time + fits edge
YOLOv8s	~22–25MB (FP16 .pt)	*Est: 350–600ms/frame (1–3 FPS)*	Better under occlusion	Too slow for Pi real-time (edge-server option later)
MobileNet-SSD (fallback)	~4–7MB (INT8 TFLite)	*Est: 80–150ms/frame (7–12 FPS)*	More misses in crowd	Backup if Pi load high

Bottleneck is vision inference → we pick smallest model that still meets <1s freshness.

Justification: Platform Choice + Trade-Off Matrix + Resource Budget

A) Platform trade-off matrix

Platform option	Why we considered	Trade-offs	Decision
RPi (Edge nodes + Edge server)	**Low** (LAN, local inference)	**Low–Med** (~5–10W/node)	Chosen (fits constraints + supports GPIO)
Mini-PC / Jetson	Low	Med–High	Optional upgrade if FPS/latency target not met
Cloud	**High/variable**	Network jitter + privacy risk if video uplink	Not in core loop (only optional non-real-time analytics)

B) Resource budget summary (expected, based on prototype + estimates)

Per-node budget

- **RPi1 (Vision + Entry PIR)**
  - **Measured:** vision loop **avg 244 ms**, **p95 307 ms** (**~4 FPS**)
  - **Estimated (from prototype run / htop):** CPU **70–95%**, RAM **1.2–1.8 GB**
  - Model: **YOLOv8n ~6–8 MB**; logs **<100 MB/day** (metadata)
- **RPi2 (Equipment PIR)**: CPU **<5%**, RAM **<150 MB**, logs minimal
- **RPi3 (MQTT + fusion + DB + dashboard)** *(Estimated)*: CPU **20–40%**, RAM **0.4–1.0 GB**
  - DB growth **~20–50 MB/day** *(basis: **~1 Hz** status updates + event messages from PIR)*

Storage footprint (overall)

- **Storage medium:** microSD (prototype) / USB SSD (Planned)
- **Format:** SQLite database + rotated JSON logs for debugging
- **Retention:** keep last 7 days (or last N rows), delete older nightly
- **RPi3** stores metadata in SQLite (time-series tables) with retention (e.g., last 7 days) and optional rotated JSON logs for debugging; no raw video is stored.

Deployment notes

- **Thermals:** use heatsink/fan on RPi1 to avoid throttling during inference
- **Scalability path:** if FPS insufficient → offload heavier model to edge server upgrade (Mini-PC/Jetson)

Scheduling Plan + Measurement Plan + Evidence + Risks

A) Scheduling plan

RPi1 (Vision + Entry PIR)

- Capture **30 FPS** → latest-frame buffer (thread)
- YOLOv8n detect+track **~4 FPS @ 480p** (skip frames if slow)
- ROI + dwell/grace anti-flicker each inference loop
- Publish **metadata only** via MQTT **1 Hz / on-change** (+ async entry PIR)

RPi3 (Edge server)

- Subscribe MQTT → **Fusion 1 Hz** (camera + entry + zones)
- DB write **batched 1–5s** (SQLite)
- WebSocket push **on-change** + alert rules (threshold + hysteresis)

RPi2 (Equipment PIR)

- Sample PIR **10 Hz** → zone state machine → publish **on-change** events

Mini timeline (one line)

- RPi1: Capture(30fps) → Infer(4fps) → ROI → Publish(1Hz)
- RPi3: Subscribe → Fuse(1Hz) → DB(batch) → WebSocket(UI)

C) Measurement plan

Metric	How you measure	Tool / method	Target
End-to-end latency (sensor→UI)	timestamps at RPi1 capture → publish → RPi3 fuse → WebSocket → UI	time.perf_counter() logs	timing logs + screenshot/graph
Inference latency / FPS	time model.track() + FPS counter	perf logs	measured table (avg, p95)
Bandwidth	bytes/sec on MQTT and REST	iftop / nload	network screenshot + payload size
CPU/RAM per node	utilization over time	htop / top	screenshots + peak notes
Offline / loss	cut internet + add RTT/loss	Wi-Fi off + tc netem	before/after logs + dashboard still works

B) Project timeline

Week 8: finalize design + baseline profiling (FPS/latency on RPi1)
Week 9: MQTT topic schema + dashboard skeleton + PIR integration
Week 10: fusion rules + DB logging + alert rules
Week 11: performance tuning (resolution/FPS, skip frames; quantize if needed) + fallback paths
Week 12: offline + degraded network tests + evidence pack + UI polish
Week 13: rehearsal + final demo + submission cleanup

Member contribution:

Jared: PIR wiring + debounce/cooldown + entry IN/OUT + zone states
Jing Chun: Offline/degraded network tests (tc netem) + buffer/replay + evidence logs/screenshots; slides
Kee Tian: RPi3 server + dashboard — MQTT broker + fusion + SQLite + REST/WebSocket; slides
Liang Jin: Integration + telemetry — MQTT topics/payload + pub/sub + end-to-end timestamps + alerts; slides
Keith: RPi1 vision — YOLOv8n tracking + ROI tuning + FPS/latency profiling; slides/video

D) Risks + mitigation

Risk	Mitigation
Model too slow / FPS drop	lower resolution/FPS, frame-skip, fallback MobileNet, optional offload to Mini-PC/Jetson
PIR noise / false triggers	debounce + cooldown + calibration
Network/LAN drop	buffer locally + replay; MQTT retain/LWT
Thermal throttling	heatsink/fan; auto back-off FPS
Storage growth	7-day retention + log rotation; DB batching

GymPulse
Edge Analytics System for Smart Condo Facilities

Video Recording Links: https://youtu.be/kG0-6D8ThlI